

2024 年非专业级别软件能力认证第一轮

(CSP-S) 入门级 C++语言模拟试题

考生注意事项:

试题纸共有 11 页，答题纸共有 1 页，满分 90 分。请在答题纸上作答，写在试题纸上的一律无效。

不得使用任何电子设备（如计算器、手机、电子词典等）或查阅任何书籍资料。

一、单项选择题（共 15 题，每题 2 分，共计 30 分；每题有且仅有一个正确选项）

1. 在 Linux 系统终端中，以下哪个命令用于显示用户当前所在目录（ D ）。

- A. cd B. ls C. ll D. pwd

2. 在图的表示中，邻接矩阵的缺点是（ A ）。

- A. 适合稠密图，空间复杂度高
B. 适合稀疏图，时间复杂度高
C. 查找邻接关系速度慢
D. 插入和删除边的操作复杂

3. 对于排序算法，说法正确的是（ D ）。

- A. 归并排序属于原地排序算法。
B. 当需要排序的数全部都不相同时，基数排序的内层排序可以选择不稳定的排序算法
C. 快速排序的时间复杂度在最坏情况下是 $O(n\log n)$
D. 堆排序没有稳定性。

4. 在 Treap 树的操作中，右旋（Right Rotation）和左旋（Left Rotation）的目的是（ B ）。

- A. 使树保持平衡 B. 维护堆的性质
C. 使得树能够分裂成两个树 D. 查询树的高度

5. 对于图的描述中，不正确的描述是（ A ）。

- A. 一个所有点都能被匹配的二分图一定是连通图
B. 欧拉图一定是连通图

- C. 强连通图不可以进行拓扑排序
- D. 把一个强连通的图的所有边都变成无向边，那么这个图一定是连通图
6. 某大型跨国连锁零售企业在世界 8 个城市共有 76 家超市，每个城市的超市数量都不相同。如果超市数量排名第四的城市有 10 家超市，那么超市数量排名最后的城市最多有几家超市（ D ）。
- A. 3 B. 4 C. 5 D. 6
7. 60 名员工投票从甲、乙、丙三人中评选最佳员工，选举时每人只能投票选举一人，得票最多的人当选。开票中途累计，前 30 张选票中，甲得 15 票，乙得 10 票，丙得 5 票。在尚未统计的选票中，甲至少再得多少票就一定当选（ B ）
- A. 15 B. 13 C. 10 D. 8
8. 在图的最小生成树算法中，Kruskal 算法使用了什么数据结构来检测环（ A ）。
- A. 并查集 B. 优先队列 C. 栈 D. 哈希表
9. 将 7 个大小相同的桔子分给 4 个小朋友，要求每个小朋友至少得到 1 个桔子，一共有几种分配方法（ C ）。
- A. 14 B. 18 C. 20 D. 22
10. 在树的高度计算中，以下哪个描述是正确的（ B ）。
- A. 树的高度是树中所有节点到根节点的最短路径
- B. 树的高度是从根节点到最深节点的路径长度
- C. 树的高度是树中节点的总数
- D. 树的高度是树中所有节点到最深节点的路径长度的平均值
11. 对于 linux 操作系统，下列描述不正确的是（ D ）。
- A. 用户层 (User Space) 包括所有运行在操作系统上的应用程序和用户进程
- B. 内核层 (Kernel Space) 负责创建、调度、终止进程和线程，提供进程间通信 (IPC) 机制。
- C. 硬件层 (Hardware) 硬件层包括计算机的物理设备和组件，如 CPU、内存、硬盘、输入输出设备等。
- D. 进程优先级是唯一影响进程调度的因素，进程优先级越高，获取的 CPU 资源越多

12. 下列问题，不可以用贪心算法解决的是（ D ）。

- A. 最小生成树问题
- B. 无负权边的最短路问题
- C. 无损数据压缩问题
- D. 网络最大流问题

13. 在图的最短路径算法中，Floyd-Warshall 算法的时间复杂度是（ A ）。

- A. $O(n^3)$
- B. $O(n^2)$
- C. $O(n \log n)$
- D. $O(n)$

14. 运行下面这段代码，当输入的两个数字是 5 和 -4 时，输出的结果是（ A ）。

```
int main() {  
    int a, b;  
    cin >> a >> b;  
    while (b) {  
        int c = a ^ b;  
        int d = (a & b) << 1;  
        a = c;  
        b = d;  
    }  
    cout << a;  
    return 0;  
}
```

- A. 1
- B. 5
- C. 9
- D. 20

15. 有若干非负整数，小明可以选择其中一个数字把他修改为另一个比原数字小的非负整数，选择下面哪个一定不可以使得这些数字的异或和为 0（ D ）。

- A. 最大的数
- B. 最小的数
- C. 和所有数字异或和相同的数字
- D. 比所有数字异或和小的数字

二、阅读程序（程序输入不超过数组或字符串定义范围；判断题正确填√，错误填×；出特殊说明外，判断题 2 分，选择题 3 分，合计 30 分）

三、

(1)

```
01 #include <iostream>
02 using namespace std;
03 void hanoi(int n, char start, char pass1, char pass2, char end) {
04     if (n > 2) hanoi(n - 2, start, end, pass2, pass1);
05     if (n > 1) printf("%c -> %c\n", start, pass2);
06     printf("%c -> %c\n", start, end);
07     if (n > 1) printf("%c -> %c\n", pass2, end);
08     if (n > 2) hanoi(n - 2, pass1, start, pass2, end);
09 }
10 int main() {
11     int n;
12     scanf("%d", &n);
13     hanoi(n, 'A', 'B', 'C', 'D');
14     return 0;
15 }
```

判断题

16. 把第 4 行到第 8 行的代码所有 "pass1" 替换为 "pass2", 所有 "pass2" 替换为 "pass1", 对于程序的正确性不会有影响 (✓)
17. 当输入的 $n=3$ 时, 会有 8 行的输出 (×)
18. 把第 3 行函数声明的所有 char 类型的参数改为 int 类型, 对于程序的正确性不会有影响 (✓)

单选题

19. 把第 5 行和第 7 行的 "if (n > 1)" 去掉, 在输入的正数 n 为 (A) 时对结果没有影响。
- A. 偶数 B. 平方数 C. 质数 D. 立方数
20. 当输入的 $n=10$ 时, 会有 (A) 行的输出
- A. 93 B. 94 C. 95 D. 96
21. 程序的时间复杂度是 (D)
- A. $O(n)$ B. $O(n \log n)$ C. $O(n^2)$ D. $O(2^n)$

答案解析：该程序是输出 4 柱汉诺塔问题的解法，第 4 行表示把前 $n-2$ 个柱子放到转移柱 1，第 5 行把第 $n-1$ 个盘子放到转移柱 2 上，第 6 行把第 n 个盘子放到目标柱上，第 7 行表示把第 $n-1$ 一个盘子从转移柱 2 移到目标柱上，第 8 表示把第 $n-1$ 个盘子放到目标柱上。

16. $\checkmark$ 相当于交换了两个中间转移柱，对输出的方案正确性没有影响。

17. $\times$ 6 行

18. $\checkmark$ 调用处是字符和输出处是“%c”，就能正常输出

19. A 每次递归调用 n 会减 2，所以偶数不会受影响

20. A, $a[2] = 3$ ，当 n 是偶数时， $a[n+2] = 2a[n]+3$;

21. D 和三柱汉诺塔复杂度只有常数级的差别

(2)

```
01 #include <iostream>
02 using namespace std;
03
04 const int maxn = 1000;
05 int n;
06 int fa[maxn], cnt[maxn];
07
08 int getRoot(int v) {
09     if (fa[v] == v) return v;
10     return getRoot(fa[v]);
11 }
12
13 int main() {
14     cin >> n;
15     for (int i = 0; i < n; ++i) {
16         fa[i] = i;
17         cnt[i] = 1;
18     }
19     int ans = 0;
20     for (int i = 0; i < n - 1; ++i) {
21         int a, b, x, y;
22         cin >> a >> b;
```

```

23         x = getRoot(a);
24         y = getRoot(b);
25         ans += cnt[x] * cnt[y];
26         fa[x] = y;
27         cnt[y] += cnt[x];
28     }
29     cout << ans << endl;
30     return 0;
31 }

```

判断题

22. 输入的 a 和 b 值应在 $[0, n-1]$ 的范围内。 (☒)
23. 第 16 行改成 $fa[i] = 0;$, 不影响程序运行结果。 (☐)
24. 若输入的 a 和 b 值均在 $[0, n-1]$ 的范围内, 则对于任意 $0 \leq i < n$ 都有 $0 \leq fa[i] < n$ 。 (☒)
25. 若输入的 a 和 b 值均在 $[0, n-1]$ 的范围内, 则对于任意 $0 \leq i < n$ 都有 $1 \leq cnt[i] \leq n$ 。 (☐)

选择题

26. 当 n 等于 50 时, 若 a, b 的值都在 $[0, 49]$ 的范围内, 且在第 25 行时 x 总是不等于 y , 那么输出为 (C)。
- A. 1276 B. 1176 C. 1225 D. 1250
27. 此程序的时间复杂度是 (C)。
- A. $O(n)$ B. $O(\log^n)$ C. $O(n^2)$ D. $O(n \log^n)$

22. 对。输入的 a, b 是集合的下标, 自然应该在 $[0, n-1]$ 之间。
23. 错。未合并前初始根节点是其本身, 为 0 显然不符合题意。
24. 对。 $fa[i]$ 是根节点, 不管在怎么合并, 根节点必然也是在 $[0, n-1]$ 之间。
25. 错。 $cnt[i]$ 是合并之后集合的元素个数, 严谨的并查集操作 $cnt[i]$ 是不会超过 n 的, 但是本题没有判断是否重复合并。所以如果先输入 $(1, 2)$, $(3, 4)$, 之后一直不断输入 $(2, 4)$, 最后 $cnt[4]$ 会超过 n 。
26. C。本题前提下 $cnt[i]$ 表示当前集合的元素个数, 每次执行都是两个集合合并, 我们可令每次都是单个集合合并进入大集合, $ans = 1*1 + 1*2 + 1*3 + \dots + 1*48 + 1*49 = 49 * (49 + 1) / 2 = 1225$ 。

27.C。getRoot 是依次查找上一个父元素，没有“压缩路径”，时间复杂度最差为 n ，所以总的时间复杂度为 n^2 。

三、完善程序（单选题，每小题 3 分、合计 30 分）

(1) (第 k 小路径) 给定一张 n 个点 m 条边的有向无环图，定点编号从 0 到 $n-1$ ，对于一条路径，我们定义“路径序列”为该路径从起点出发依次经过的顶点编号构成的序列。求所有至少包含一个点的简单路径中，“路径序列”字典序第 k 小的路径。保证存在至少 k 条路径。上述参数满足 $1 \leq n, m \leq 10^5, 1 \leq k \leq 10^{18}$ 。

在程序中，我们求出从每个点出发的路径数量。超过 10^{18} 的数都用 10^{18} 表示。然后我们根据 k 的值和每个顶点的路径数量，确定路径的起点，然后可以类似地依次求出路径中的每个点。

试补全程序。

```
01 #include <iostream>
02 #include <algorithm>
03 #include <vector>
04
05 const int MAXN = 100000;
06 const long long LIM = 100000000000000000011;
07
08 int n, m, deg[MAXN];
09 std::vector<int> E[MAXN];
10 long long k, f[MAXN];
11
12 int next(std::vector<int> cand, long long &k) {
13     std::sort(cand.begin(), cand.end());
14     for (int u : cand) {
15         if (①) return u;
16         k -= f[u];
17     }
18     return -1;
```

```

19 }
20
21 int main() {
22     std::cin >> n >> m >> k;
23     for (int i = 0; i < m; ++i) {
24         int u, v;
25         std::cin >> u >> v;
26         E[u].push_back(v);
27         ++deg[v];
28     }
29     std::vector<int> Q;
30     for (int i = 0; i < n; ++i)
31         if (!deg[i]) Q.push_back(i);
32     for (int i = 0; i < n; ++i) {
33         int u = Q[i];
34         for (int v : E[u]) {
35             if (②)
36                 Q.push_back(v);
37             --deg[v];
38         }
39     }
40     std::reverse(Q.begin(), Q.end());
41     for (int u : Q) {
42         f[u] = 1;
43         for (int v : E[u])
44             f[u] = ③;
45     }
46     int u = next(Q, k);
47     std::cout << u << std::endl;
48     while (④) {
49         ⑤;
50         u = next(E[u], k);
51         std::cout << u << std::endl;
52     }
53     return 0;

```


54 }

28. ①处应填 (B)

- A. $k \geq f[u]$
- B. $k \leq f[u]$
- C. $k > f[u]$
- D. $k < f[u]$

29. ②处应填 (A)

- A. $\text{deg}[v] == 1$
- B. $\text{deg}[v] == 0$
- C. $\text{deg}[v] > 1$
- D. $\text{deg}[v] > 0$

30. ③处应填 (A)

- A. $\text{std::min}(f[u] + f[v], \text{LIM})$
- B. $\text{std::min}(f[u] + f[v] + 1, \text{LIM})$
- C. $\text{std::min}(f[u] * f[v], \text{LIM})$
- D. $\text{std::min}(f[u] * (f[v] + 1), \text{LIM})$

31. ④处应填 (D)

- A. $u \neq -1$
- B. $!E[u].\text{empty}()$
- C. $k > 0$
- D. $k > 1$

32. ⑤处应填 (C)

- A. $K+=f[u]$
- B. $k-=f[u]$
- C. $--k$
- D. $++k$

答案解析:

28. 答案:B,next 函数是在 cand 队列中查找第 k 条在哪个结点为起点的路径中, 所以如果 $k \leq f[u]$, 说明第 k 小就在 u 为起点的路径中, 否则 $k=f[u]$, 找下一个结点。

29. 答案:A, 入度为 0 的点入队列, $--\text{deg}[v]$ 在入队列的后面执行, 所以这里判

断 $\deg[v]=1$; 题目保证了是 DAG, 所以整个队列中一定会有 n 个结点。

30. 答案:A, 动态规划, 计算 u 为起点的路径树, $f[u]=1$ 是只有自身的一条路, 枚举所有子结点的路径数, 累加。先把 Q 逆序是因为要拓扑序靠前的结点计算依赖拓扑序靠后的结点。

31. 答案:D, next 函数执行后我们知道了第 k 小的路径在 u 开头的路径中, 当 $k=1$ 时, 这条路径就只有 u 一个结点, 如果 $k>1$, 则需要从 u 的子结点中继续找。

32. 答案:C, $-k$ 表示减去 u 为唯一结点的那条路径。

(2) (归并第 k 小) 已知两个长度均为 n 的有序数组 $a1$ 和 $a2$ (均为递增序, 但不保证严格单调递增), 并且给定正整数 k ($1 \leq k \leq 2n$), 求数组 $a1$ 和 $a2$ 归并排序后的数组里第 k 小的数值。

试补全程序。

```
01 #include <bits/stdc++.h>
02 using namespace std;
03
04 int solve(int *a1, int *a2, int n, int k) {
05     int left1 = 0, right1 = n - 1;
06     int left2 = 0, right2 = n - 1;
07     while (left1 <= right1 && left2 <= right2) {
08         int m1 = (left1 + right1) >> 1;
09         int m2 = (left2 + right2) >> 1;
10         int cnt = ①;
11         if (②) {
12             if (cnt < k) left1 = m1 + 1;
13             else right2 = m2 - 1;
14         } else {
15             if (cnt < k) left2 = m2 + 1;
16             else right1 = m1 - 1;
17         }
18     }
19     if (③) {
20         if (left1 == 0) {
21             return a2[k - 1];
22         } else {
```

```
23         int x = a1[left1 - 1], ④;  
24         return std::max(x, y);  
25     }  
26 } else {  
27     if (left2 == 0) {  
28         return a1[k - 1];  
29     } else {  
30         int x = a2[left2 - 1], ⑤;  
31         return std::max(x, y);  
32     }  
33 }  
34 }
```

33. ①处应填 (D)

- A. $(m1 + m2) * 2$
- B. $(m1 - 1) + (m2 - 1)$
- C. $m1 + m2$
- D. $(m1 + 1) + (m2 + 1)$

34. ②处应填 (B)

- A. $a1[m1] == a2[m2]$
- B. $a1[m1] <= a2[m2]$
- C. $a1[m1] >= a2[m2]$
- D. $a1[m1] != a2[m2]$

35. ③处应填 (C)

- A. $left1 == right1$
- B. $left1 < right1$
- C. $left1 > right1$
- D. $left1 != right1$

36. ④处应填 (C)

- A. $y = a1[k - left2 - 1]$
- B. $y = a1[k - left2]$
- C. $y = a2[k - left1 - 1]$
- D. $y = a2[k - left1]$

37. ⑤处应填 (A)

- A. $y = a1[k - left2 - 1]$

- B. $y = a1[k - left2]$
- C. $y = a2[k - left1 - 1]$
- D. $y = a2[k - left1]$

答案解析：

33. 【答案】D 【解析】首先要考虑二分的目的，二分是为了确定 $a1$ 和 $a2$ 中哪些部分是属于合并后前 k 个数字的。所以 cnt 代表的是 $a1[0..m1]$ 和 $a2[0..m2]$ 中数字的个数，需要考虑到数组下标是从 0 开始的。

34. 【答案】B 【解析】观察第 12、13 行，如果 $cnt < k$ ，程序舍弃了 $a1[left1..m1]$ 这一段如果 $cnt \geq k$ ，程序舍弃了 $a2[m2..right2]$ 这一段。说明此时 $a1[left1..m1]$ 在归并之后一定在 $a2[m2..right2]$ 前面，那么 $a1[m1]$ 必须小于等于 $a2[m2]$ 。

35. 【答案】C 【解析】在第 7 行，循环结束的条件是 $left1 > right$ 或者 $left2 > right2$ 此时 if 语句是将这两种情况分类。

36. 【答案】C 【解析】如果 $left1$ 不等于 0，那么 $a1[0..left1-1]$ 都在合并后前 k 个数字以内然后 $a2$ 的前 $k-left$ 个数字也是合并后的前 k 个数字以内，所以最终答案是 $a1[left1-1]$ 与 $a2[k-left-1]$ 的最大值。

37. 【答案】A 【解析】此处和第④处是相同的，当 $left2$ 不等于 0 时，前 k 个数是 $a2[0..left2-1]$ 和 $a1[0..k-left-1]$ 给并后的结果。